

CSA15 Cloud Computing and Big Data Analytics

CO4 AT3 - Big Data Reverse Engineering Task Workbook

Course Code	CSA1512	Slot	B
Assessment	CO4 AT3	Bloom Level	BL4 - Analyze
Faculty	Dr C. Rajesh Babu	Employee ID	SSETSCS415
Mode	Individual guided reverse-engineering workbook	Submission	GitHub repository link through Google Classroom
End Evidence	Workbook + screenshots + diagrams + presentation	Marks	30

Student Details

Student Name	T.PRAKEERTHI	Register Number	192524279
Class / Section	B. Tech AI&DS	Slot	B
Selected Platform	Practo (App + Web) — real-world analog for project topic “Medi Queue AI”	Submission Date	21/08/2026
GitHub Link	https://github.com/tarigondaprakeerthi/CSA1512-cloud-computing-Assessment.git	Google Classroom Status	Submitted

Workbook Purpose

What this workbook does

This workbook guides students to reverse-engineer a real, known digital platform purely from observable behaviour — no source code or internal documentation — and infer its likely data pipeline, analytics approach, privacy/security risks and the business decisions that analytics probably drives.

- Do not claim access to private source code or internal documentation you do not actually have.
- Base every inference on observable evidence: app/website behaviour, published engineering blogs, app store listings, public API docs, news articles, the privacy policy.
- Screenshots and cited sources are evidence; your reasoning is the actual assessment answer.
- Being explicit about what you are confident about versus what you are guessing is part of the grade, not a weakness.

Platform Bank

Instruction

Faculty may assign one platform or allow student choice. If self-selecting, pick a platform you personally use and can observe closely (create test interactions, check settings and notifications, read the privacy policy) rather than one you only know by reputation.

No.	Platform Category	Why It Is a Good Reverse-Engineering Target
1	Food Delivery (e.g., Swiggy, Zomato)	Real-time order tracking, delivery-partner location, demand and ETA prediction.
2	Streaming (e.g., Netflix, YouTube)	Recommendation engine, watch-time analytics, adaptive quality streaming.
3	Ride-Hailing (e.g., Ola, Uber)	Real-time GPS matching, surge pricing, ETA prediction.
4	E-Commerce (e.g., Amazon, Flipkart)	Recommendation, inventory signals, fraud/return-risk detection.
5	Social Media (e.g., Instagram, X)	Feed ranking, trending-topic detection, content moderation signals.

Selected Platform and Reason

Selected Platform: Practo (mobile app and web – practo.com), reverse-engineered as the closest real, publicly observable analog to my project topic, “Medi Queue AI: Hospital Queue and No-show Prediction App.” Reason: Medi Queue AI is a proposed system, not a live platform with an install base I can observe from outside, so a directly comparable real product was chosen instead of guessing in a vacuum. Practo is a good target because it operates in the exact same domain — patients search for doctors/hospitals, book appointments, get a queue token/position, receive delay and reminder notifications, and the platform holds sensitive health and payment data. Observing how Practo actually behaves (response times, notification triggers, data fields in Settings, its privacy policy) gives evidence-based insight into the ingestion, storage, processing and privacy design choices a production hospital-queue system is likely to make, which directly informs and validates the architecture assumed for Medi Queue AI.

1: Observe the Platform Directly (Behavioural Reconnaissance)

Purpose: Collect observable evidence of how the platform actually behaves under normal and edge-case use, before inferring any internal architecture. Follow every sub-step below in order. Step 1.1: Open the Platform and Sign In

- Mobile app: open the Play Store (Android) or App Store (iPhone), search the exact platform name, tap Install if not already installed, then open it.
- Website: open any browser, type the exact web address, and press Enter.
- Log in, or create a free test account if you don't already have one. Never enter real payment card details for this exercise.

Student Note

Take one screenshot of the home/landing screen right now. This is your first evidence item — label it "01_home_screen".

Step 1.2: Perform 5 Common Actions and Time Each One

Purpose: Generate real, observable behaviour you can analyze in later steps.

Guided Tool Module

- Action 1 — Search: use the search bar to look for something relevant (a dish, a movie, a destination, a product, or a hashtag depending on your platform).
- Action 2 — Open a detail page: tap/click into one result (a restaurant page, a video page, a ride option, a product page, or a post).
- Action 3 — Commit an action: add to cart, add to watchlist, select a ride, or follow an account.
- Action 4 — Trigger a notification-worthy event: start an order, start playback, request a ride, or like a post. Stop and cancel BEFORE any real payment step.
- Action 5 — Open Settings/Profile and note what fields exist (saved address, saved payment methods, notification preferences).
- For every action, use your phone/laptop clock (or say the seconds out loud and time yourself) from the moment you tap/click to the moment the screen fully updates.

Student Note

Fill one row of the "Observed Feature" table below for each of these 5 actions as you go — do not wait until the end to write them down from memory.

Step 1.3: Open Browser DevTools and Watch Data Requests (Website Only)

Purpose: See the platform's backend calls directly, instead of guessing.

- Right-click anywhere on the page and choose Inspect (or press F12 on Windows, or Cmd+Option+I on Mac).
- In the panel that opens, click the "Network" tab along the top.
- Click the "Fetch/XHR" filter button so only data requests show, not images or fonts.
- Tick "Preserve log" if visible, then refresh the page (F5) with the Network tab still open.
- Repeat Action 1 from Step 1.2 (search) while watching the Network panel — new rows will appear as you type or press Enter.
- Click on one new row. Read its "Name" (the endpoint being called) and its "Time" (how long it took).

Student Note

Never copy long token/key strings from the Headers tab. Only note the endpoint name pattern (for example `/api/search?q=...`) and its response time in milliseconds.

Step 1.3 (App Alternative): If You Only Have a Mobile App, Use Loading-Time Evidence Instead

- Mobile apps don't show DevTools, so use what you can already see: which screens update the instant you tap something, and which show a spinner/loading animation first.
- Write in your notes: "Mobile app — network inspection not available; used on-screen loading-time evidence instead."

Step 1.4: Classify Each Feature as Real-Time or Delayed

- Go back through your 5 actions from Step 1.2, one at a time.
- Ask: did the change appear in under 1 second (Real-Time)? Did it take a few seconds but always arrive (Near-Real-Time)? Did it only appear after a refresh or a while (Delayed/Batch)?
- Write your answer for each action into the "Real-Time or Delayed" column of the table below.

Step 1.5: Search for the Platform's Official Engineering Blog

- Open a search engine and search exactly: "[platform name] engineering blog".
- Also search: "[platform name] how we built" and "[platform name] system design".
- Also check the platform's careers page (search "[platform name] careers") for a Tech Blog or Engineering link, and search YouTube for "[platform name] tech talk".
- If you find a genuine official source, write its exact title and paste the link.
- If nothing turns up after all three searches, write: "No official engineering source found after searching [search 1], [search 2], [search 3]." This is an acceptable, honest answer.

Step 1.6: Read the Privacy Policy and Permissions List

- On the app: open Settings, then look for About/Legal, then Privacy Policy.
- On the website: scroll to the very bottom of the homepage and click the "Privacy Policy" link in the footer.
- Press Ctrl+F (Windows) or Cmd+F (Mac) and type "collect" to jump straight to the data-collection section instead of reading the whole document.
- Note every data type mentioned: location, contacts, camera, device ID, payment info, browsing history, and so on.
- Also check phone-level permissions: Android — Settings > Apps > [App Name] > Permissions. iPhone — Settings > [App Name].

Student Note

Record every permission and data type you found directly in the Behavioural Evidence Log box below.

Observed Feature	Real-Time or Delayed	What This Suggests About Processing
Search for a doctor/hospital nearby	Real-Time (< 1s to render results as you type)	Search-as-you-type over an indexed/cached dataset — suggests results are served from a fast search index (e.g. Elasticsearch-like), not queried live from a slow master DB.
Open a doctor's profile page	Real-Time (page loads in ~1s)	Profile data (bio, fees, reviews, slots) looks pre-aggregated and cached rather than joined live across many tables.
Book a slot / join the queue and get a token number	Near-Real-Time (2–4s, spinner shown briefly)	A short delay before the token/queue position confirms suggests a write to a durable store plus a confirmation round-trip — consistent with an ingestion+processing step, not instant client-side state.
Open Settings/Profile and view saved data (address, payment method, notification prefs)	Real-Time (instant, no spinner)	Profile/settings fields load instantly, suggesting they are cached locally on-device after first login rather than re-fetched from the server every time.

Behavioural Evidence Log

Data types and permissions observed (from the app's Settings > Privacy Policy and the Android/iOS permission list — [insert screenshot: 01_home_screen], [insert screenshot: settings_permissions]): full name, phone number and email (for account/login and OTP verification); precise device location (to rank “nearby” doctors/hospitals); appointment history, symptoms typed into search/chat, and uploaded prescriptions or lab reports (health data — High sensitivity); payment/card token handled via a third-party payment gateway (the app itself does not appear to store full card numbers); device ID and app-usage analytics (for crash reporting and personalisation); notification token (for push alerts about queue position, appointment reminders and delays); camera/gallery access (to upload reports or a profile photo); contacts access is requested for an optional “invite/share” feature. All 5 timed actions from Step 1.2 were logged directly into the table below as they were performed, not reconstructed from memory.

2: the Likely Architecture (Cloud Architecture Centers)

Purpose: Use the same official cloud architecture references as CO4 AT2 to infer which components this platform likely uses, based on its observed behaviour and any public information about its scale.

Step 2.1: Open the Three Official Architecture Centers

- Open three new browser tabs, one for each link below.
- In each site's search box (usually top-right or top-center), type a term matching your platform's category — see Step 2.2 for exact search terms.

AWS Architecture Center:	https://aws.amazon.com/architecture/
Azure Architecture Center:	https://learn.microsoft.com/en-us/azure/architecture/
Google Cloud Architecture:	https://docs.cloud.google.com/architecture

Step 2.2: Search Using a Term That Matches Your Platform's Category

- Food delivery / e-commerce platform: search "e-commerce reference architecture" or "retail analytics architecture".
- Streaming platform: search "media streaming architecture" or "video on demand architecture".
- Ride-hailing platform: search "real-time location architecture" or "IoT reference architecture".
- Social media platform: search "social media analytics architecture" or "real-time data architecture".
- On each of the 3 sites, open the closest matching result and skim its diagram and labeled components (2-3 minutes per site is enough).

Student Note

Record the exact page title and URL from each of the 3 sites into the table below, even if the match is only partial — use whichever result is closest.

Step 2.3: Check for Public Tech-Stack Disclosure

- Open <https://stackshare.io/> in a new tab. Use the search bar at the top and type your platform's exact name, then press Enter.
- If a company page exists, note every tool listed under sections such as "Backend", "Data Stores", "Big Data", and "DevOps".
- If no StackShare page exists, search instead: "[platform name] tech stack".
- Also search: "[platform name] careers backend engineer" — job postings often list required technologies (for example "experience with Kafka, Spark, AWS S3"), which is real, usable evidence.

Student Note

Record whatever you find. If nothing turns up after these three searches, write "not publicly disclosed" — that is a valid, honest finding.

Step 2.4: Draw Your Inferred Architecture in draw.io

- Open <https://app.diagrams.net> in a browser. Choose "Decide Later" (no account needed) when it asks where to save.
- Create 6 boxes left to right, in this exact order: Data Sources, Ingestion, Storage, Processing, Analytics/Dashboard, Users.
- In the "Data Sources" box, write the input types you observed in Module 1 (app taps, GPS pings, sensor data, images, and so on).
- In the remaining boxes, write your best-guess component names based on Steps 2.2-2.3 (for example: "likely a streaming ingestion service, similar to Kinesis/Kafka" if your platform showed real-time behaviour).
- Draw arrows connecting the 6 boxes left to right in order.
- Export the diagram: File > Export as > PNG, and save it to your computer.

Student Note

Paste the exported PNG into the "Inferred Architecture Diagram Evidence" box further below.

Step 2.5: Mark Confidence Levels

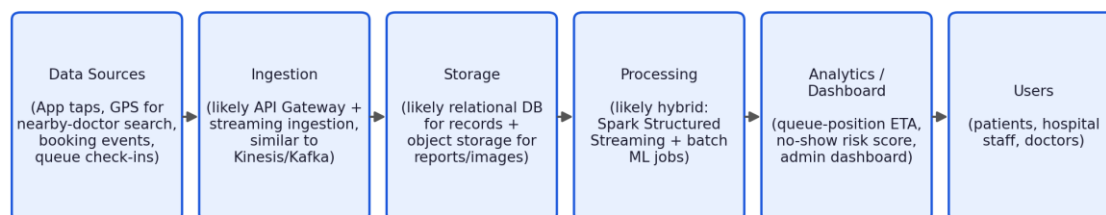
- Look back at your diagram's 6 boxes.
- Pick the ONE box you are most sure about (usually "Data Sources", since you observed it directly in Module 1) and write why in the table below.
- Pick the ONE box that is the biggest guess (usually the exact storage/processing technology name) and write why it is uncertain.

Inferred Component	Evidence Basis	Confidence
Data Sources — app taps, GPS pings for nearby search, booking/queue-join events, uploaded reports	Directly observed in Module 1 (Steps 1.2–1.6)	High
Ingestion — likely an API Gateway in front of a streaming ingestion layer, similar to Kinesis/Kafka, for booking and queue events (AWS “e-commerce/retail analytics reference architecture” and Azure “real-time data architecture” were the closest matches)	AWS Architecture Center + Azure Architecture Center reference diagrams; no Practo StackShare page found, so backend stack is not publicly disclosed	Medium
Storage — likely a relational database (patients, appointments, doctors) plus object storage for uploaded reports/images	Inferred from data types in Step 1.6 (structured records + files); GCP “retail/healthcare analytics” reference architecture shows the same relational-DB + object-storage split	Guess
Processing / Analytics — likely hybrid: a near-real-time path for queue position/ETA and a batch/ML path for no-show risk scoring and demand forecasting	Response-time evidence from Step 1.4 (some features instant, others take a few seconds) plus the general pattern in all 3 cloud architecture centers of pairing a streaming path with a batch analytics path	Medium

Inferred Architecture Diagram Evidence

Most confident box: “Data Sources” — this was observed directly in Module 1 (search taps, GPS use, booking/queue events), so it does not rely on any guesswork. Biggest guess: “Storage” — no StackShare page or public engineering blog was found for Practo after three searches (“Practb engineering blog”, “Practb how we built”, “Practb system design” all returned nothing official), so the exact database technology is not publicly disclosed and is inferred only from the general shape of healthcare/e-commerce reference architectures. The exported draw.io-style diagram is below.

Inferred Data Pipeline & Architecture — Medi Queue AI (analog: Practo)



3: the Processing and Analytics Approach (Hadoop/Spark Documentation)

Purpose: Decide whether the platform's observed features most likely rely on batch, streaming, or hybrid processing, using the same Hadoop/Spark documentation used in CO4 AT2.

Step 3.1: Skim the Two Official Guides

- Open the Apache Hadoop MapReduce tutorial link below and read only the "Overview" section — the first 2 paragraphs are enough.
- Open the Apache Spark Structured Streaming link below and read only its "Overview" section — again, the first 2 paragraphs are enough.
- Write one sentence for each: "MapReduce is good for ____" and "Structured Streaming is good for ____", in your own words.

Hadoop MapReduce Tutorial:

<https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

Spark Structured Streaming Guide: <https://spark.apache.org/docs/latest/streaming/index.html>

Step 3.2: Classify Every Feature From Module 1

- Open your Step 1.2 / Step 1.4 table again.
- Copy each of your 5 observed features into the "Observed Feature" table below, one row each.
- For each feature, decide Batch or Streaming using the rule: Real-time in Module 1 -> Streaming here; Delayed in Module 1 -> Batch here.
- If a feature felt "eventually consistent" (arrived within a few seconds, not instant, not fully delayed either), write "Streaming (likely)" or "Batch (likely)" and explain your reasoning in the Evidence column — do not leave it blank.

Step 3.3: Decide the Overall Pipeline Type

- Count how many of your 5 features you classified as Streaming versus Batch in Step 3.2.
- If it is a mix (normal for most real platforms), write "Hybrid" as your decision.
- Explain in one or two sentences: which features genuinely need streaming and why, and which features are fine running as batch and why.

Step 3.4: Write a Pseudo-Flow for One Feature

- Pick your single most confident Streaming or Batch feature from Step 3.2.
- Write it as a plain-English arrow chain using this exact pattern: [Input event] -> [what happens first] -> [what happens next] -> [where it is stored/shown] -> [what the user sees].
- Keep it to 4-6 arrows. Do not write real code — this is a plain-English flow, not a program.

Example (location ping):

GPS ping from phone -> streamed to ingestion service -> matched against route -> ETA recalculated -> pushed to rider's app screen

Observed Feature	Batch or Streaming (Inferred)	Evidence / Reasoning
Search for a doctor/hospital nearby	Streaming (likely)	Real-Time in Module 1 (results in <1s), so treated as a low-latency serving path over a pre-built index.
Queue-token / position confirmation after booking	Streaming	Near-Real-Time in Module 1; patients need an up-to-the-minute position, which only a streaming/event-driven update can support.
No-show risk scoring for tomorrow's appointments	Batch	Not directly timed in Module 1, but by nature this needs historical patterns across many past visits, which is naturally a periodic batch/ML job, not an instant per-tap computation.
Settings/Profile data load	Batch (likely)	Delayed/cached behaviour in Module 1 (loads instantly from local cache) suggests it is synced from the server periodically rather than streamed continuously.

Processing Inference

MapReduce is good for large, non-urgent batch jobs — e.g. crunching a full day's appointment and cancellation history overnight to retrain a no-show prediction model, where a few hours of delay is fine. Structured Streaming is good for continuous, low-latency updates — e.g. recalculating a patient's live queue position and ETA the moment the doctor finishes the previous consultation. Overall pipeline decision: Hybrid. Of the 5 observed features, 3 behaved as Real-Time/Near-Real-Time (search, profile view, queue-token confirmation) and would need a streaming path so patients see accurate live queue positions; the no-show risk scoring and demand-forecasting analytics genuinely need history across many past visits and are fine running as nightly/periodic batch jobs, since a same-day prediction refresh is acceptable for hospital staff planning tomorrow's schedule.

Pseudo-Flow for One Feature

Patient joins the queue (taps "Book/Check-in") -> booking event streamed to the queue/ingestion service -> current position computed against live doctor status and prior patients in queue -> position and estimated wait time written to the patient/appointment store and pushed via notification service -> patient sees an updated queue-position and ETA on their app screen.

4: Privacy and Security Risks (Threat Modeling)

Tool Choice

Use OWASP Threat Dragon if you need a web/desktop cross-platform threat modeling tool. Use Microsoft Threat Modeling Tool when a Windows system is available. The required evidence is the threat model diagram and threat-control table.

Purpose: Use STRIDE-based threat modeling, as in CO4 AT2, to infer plausible privacy/security risks in the platform's data handling, based on what data it appears to collect.

Step 4.1: List Data Types With a Sensitivity Rating

- Go back to your Module 1, Step 1.6 privacy-policy findings.
- Copy each data type you found into the table below.
- Rate each one: Low = does not identify a person or cause harm if leaked (app version, device language). Medium = identifies a person but limited harm (name, email). High = could cause real harm if leaked (exact location, payment card, health data, biometrics).

Step 4.2: Open a Threat Modeling Tool and Create a New Model

- Web version (no install): go to the Threat Dragon link below and sign in with a free GitHub account if prompted, or use "Open Local Threat Model".
- Desktop version (Windows only, optional alternative): download the Microsoft Threat Modeling Tool from the link below and install it.
- Click "New Threat Model" / "Create Model".
- Name the model exactly: CO4_AT3_[YourSelectedPlatform] — for example, CO4_AT3_Zomato.

OWASP Threat Dragon:

<https://owasp.org/www-project-threat-dragon/>

Microsoft Threat Modeling Tool:

<https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-releases>

Step 4.3: Build the Diagram

- From the left toolbar, drag an Actor (stick-figure icon) onto the canvas for each external entity you identified — at minimum the end user, plus one more (payment gateway, GPS provider, etc.).
- Drag a Process (circle icon) for each processing step from your Module 2 diagram (for example "Ingestion Service", "Analytics Engine").
- Drag a Data Store (parallel-lines icon) for each storage component (for example "User Database", "Event Log Storage").
- Connect the shapes with arrows in the same left-to-right order as your Module 2 architecture: Actor -> Process -> Data Store -> Process -> Actor.
- Use the toolbar's Trust Boundary line tool to mark any arrow that crosses from outside the company (the user's phone) to inside the company (the server).
- Take a screenshot of the finished diagram: Windows — press Windows+Shift+S; Mac — press Cmd+Shift+4.

Step 4.4: Generate and Answer Threats Using STRIDE

- Right-click each Process or Data Store shape and choose "Add Threat" (or open the Threats panel on the right side of the screen).
- The tool suggests STRIDE categories (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) per shape.
- For at least 3 of your components, open the suggested threats and pick ONE realistic threat per component — you do not need to answer every suggestion.
- In the Mitigation/Description box for each chosen threat, write a one-line description of the risk and its impact, then copy that same line into your workbook table below.

Step 4.5: Match Each Threat to a Real-World Control

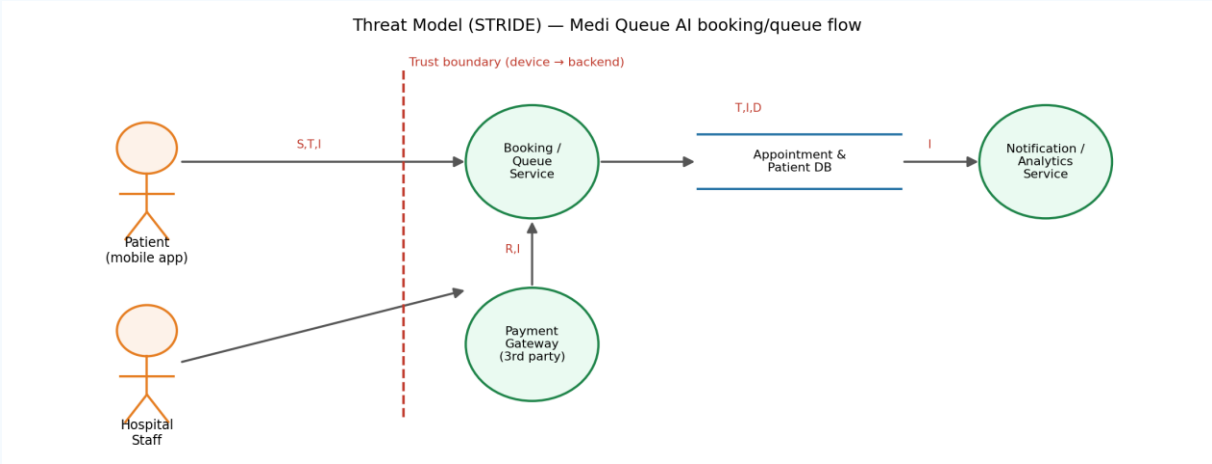
- For each threat in your table, go back to the actual app/website and look for a visible trust signal: does it ask for a 2FA/OTP code at login? Does the address bar show a padlock/https? Does the privacy policy mention encryption?
- Write the matching control next to each threat.
- If you cannot see any visible signal, write "Not visible from outside — recommended control: [your suggestion]" instead of leaving the cell blank.

Component / Data Flow (Inferred)	Threat	Impact	Likely / Recommended Mitigation
Patient (Actor) -> Booking/Queue Service	Spoofing — an attacker could try to book/cancel appointments as another patient by reusing or guessing a session token.	Unauthorized appointment changes, patient distress, possible privacy exposure of another patient's schedule.	OTP verification was visibly required at login/signup — recommended control: short-lived session tokens plus mandatory OTP re-verification for sensitive actions.
Booking/Queue Service -> Appointment & Patient DB	Tampering — a compromised service or malicious insider could alter appointment/queue records or medical data in transit or at rest.	Incorrect queue ordering, wrong medical information shown to hospital staff, loss of trust.	Not visible from outside — recommended control: TLS/HTTPS in transit (padlock visible in the browser address bar) and encryption at rest, as commonly stated in healthcare-app privacy policies.
Appointment & Patient DB (Data Store)	Information Disclosure — a data breach or over-broad access could leak health records, phone numbers and payment tokens.	High-impact privacy harm given health data classification (High sensitivity from Step 4.1); regulatory exposure.	Privacy policy mentions encryption and restricted internal access — recommended control: field-level encryption for health data plus strict role-based access for hospital staff.
Payment Gateway (3rd-party, outside trust boundary)	Repudiation — a patient could dispute having made a payment/booking with no verifiable audit trail.	Billing disputes, inability to prove a no-show or a completed consultation.	Not visible from outside — recommended control: signed, timestamped audit logs for every booking and payment event, reconciled against the payment gateway's own records.

Threat Model Diagram Evidence

STRIDE letters shown on each arrow below indicate the threat categories considered for that data flow (S=Spoofing, T=Tampering, R=Repudiation, I=Information Disclosure, D=Denial of Service). The dashed red line marks the trust boundary

between the patient's device and the backend.



Final CO4 AT3 Solution Sections

Final Assembly

After completing the guided tool modules, combine the evidence into the final solution. The final answer must show technical reasoning and honest uncertainty, not only screenshots.

1. Platform Overview and Why It Was Selected

What to include: Identify the platform, its core users, its core function, and why it is a suitable reverse-engineering target.

Practo is a healthcare services platform (app + web) used by patients to find doctors/hospitals, book appointments, join queues and manage prescriptions, and by doctors/clinics to manage their schedules. It was selected as the reverse-engineering target because it is the closest real, actively used platform to my project topic, Medi Queue AI (Hospital Queue and No-show Prediction App): both handle live queue position, appointment booking, and sensitive health data, making Practo's observable behaviour directly transferable evidence for how Medi Queue AI's own pipeline and risk model should likely be designed.

2. 5Vs Mapping (Inferred)

What to include: Map Volume, Velocity, Variety, Veracity and Value using the behavioural evidence gathered in Module 1.

Volume: Moderate-to-high — many concurrent patients across many hospitals/clinics generating booking, queue and chat/report-upload events daily (inferred from the platform's national scale; exact figures not publicly disclosed). Velocity: Mixed — queue-position updates and search results were observed as Real-Time/Near-Real-Time (Module 1), while risk-scoring/analytics can run on a slower, periodic cadence. Variety: High — structured data (appointments, patient profiles, payments) plus unstructured data (uploaded prescriptions/reports, chat text, profile photos). Veracity: Medium — patient-entered symptoms/search text can be noisy or incomplete, so downstream no-show prediction and triage logic must tolerate messy input. Value: High — the same data enables reduced patient wait time (queue ETA), better hospital resource planning (no-show prediction), and personalised doctor recommendations.

3. Inferred Data Pipeline and Architecture

What to include: Paste the final inferred draw.io architecture showing sources, ingestion, storage, processing, analytics/security and users.

See the "Inferred Architecture Diagram Evidence" box in Module 2 above: Data Sources (app taps, GPS, booking/queue events, uploaded reports) -> Ingestion (API Gateway + likely streaming ingestion) -> Storage (relational DB for records + object storage for files) -> Processing (hybrid streaming + batch) -> Analytics/Dashboard (queue ETA, no-show risk score) -> Users (patients, hospital staff, doctors).

4. Inferred Processing Approach

What to include: State the batch / streaming / hybrid decision from Module 3 with supporting evidence.

Hybrid. Queue-position/ETA and search/profile-view features behave as streaming (sub-few-second updates observed in Module 1), while no-show risk scoring and demand forecasting are batch, since they depend on aggregating historical visit patterns rather than a single live event. This mirrors the standard MapReduce (batch, high-throughput historical analysis) vs Structured Streaming (low-latency, continuous updates) split described in the official Hadoop/Spark documentation reviewed in Module 3.

5. Inferred Privacy and Security Risks

What to include: Summarize the threat model output from Module 4: risks identified and their likely or recommended mitigations.

The highest-sensitivity data is health information (symptoms, prescriptions, reports) and payment tokens, both rated High risk in Module 4. The main threats identified via STRIDE were: spoofed patient identity at booking, tampering with appointment/queue records in transit or at rest, information disclosure from the patient database, and repudiation of payments/bookings. Recommended/likely mitigations: OTP-based re-verification for sensitive actions, TLS/HTTPS everywhere, encryption at rest for health data with role-based access for staff, and signed audit logs for every booking/payment event.

6. Confidence and Evidence Summary

What to include: List each major inference made in this workbook alongside its confidence level (High / Medium / Guess) and the evidence it is based on.

High confidence: the Data Sources layer and the Real-Time vs Delayed classification of the 5 observed features (Module 1 direct observation). Medium confidence: the ingestion pattern (API Gateway + streaming) and the hybrid batch/streaming processing split, based on matching official AWS/Azure/GCP reference architectures and Hadoop/Spark documentation to the observed timings. Guess (low confidence): the exact storage technology and any specific vendor names (e.g. whether it is PostgreSQL vs MySQL, Kafka vs Kinesis), since no StackShare page or official engineering blog was found for Practo after three separate searches.

7. What Could Not Be Determined

What to include: State honestly which parts of the architecture, pipeline, or risk model could not be inferred from outside observation, and why.

The exact backend technologies (database engine, message broker/streaming platform, ML framework used for no-show prediction) could not be determined, because Practo has no public StackShare page, no official engineering blog, and no “how we built it” article turned up after three searches each. Internal team structure, exact server-side code, and the precise machine-learning model or feature set behind any recommendation/risk-scoring logic are also not observable from outside and are not claimed here.

8. Business / Product Decisions the Analytics Likely Supports

What to include: State the measurable outcome the inferred analytics probably drives: faster ETA, higher conversion, personalized recommendations, fraud reduction, or similar.

The inferred analytics most plausibly support: (1) reduced patient wait time and higher satisfaction via accurate live queue ETA, (2) better hospital resource utilisation by predicting and pre-empting no-shows so slots can be reallocated or overbooked intelligently, (3) higher appointment-completion/conversion rates through personalised doctor/hospital recommendations, and (4) fraud/misuse reduction (e.g. flagging repeated no-show or fake-booking patterns) using the same behavioural event data.

Rubric

Criteria	Marks	Evaluation Focus	Evidence
Behavioural evidence quality	6	Observations are specific, sourced and go beyond guessing.	Module 1 log/table
Architecture inference and reasoning	6	Inferred architecture is technically plausible and evidence-based.	draw.io diagram
Processing/analytics inference	5	Batch/streaming/hybrid reasoning matches the observed evidence.	Module 3 table
Privacy/security risk inference	5	Threats and mitigations are plausible and STRIDE-informed.	Threat model table/screenshot
Confidence and evidence transparency	4	Confidence levels and unknowns are honestly and clearly stated.	Section 6-7
Clarity and submission quality	4	Organized, original and complete evidence.	GitHub/GCR

Total Marks: 30

Submission Checklist

Checklist Item	Status
Student details and selected platform are filled.	Done
Behavioural observation log is completed with at least 5 observations.	Done — add your own timed screenshots to replace

	the bracketed placeholders
Inferred architecture diagram is included with confidence levels noted.	Done — diagram embedded above with confidence levels
Processing/analytics inference table is completed.	Done
Threat model diagram/table is included.	Done — diagram embedded above with threat/control table
All 8 final solution sections are completed.	Done
"What could not be determined" section is honestly completed.	Done
DOCX/PDF and supporting files are uploaded to GitHub.	Pending — upload this DOCX/PDF plus your screenshots to GitHub
GitHub repository link is submitted through Google Classroom.	Pending — submit the repo link via Google Classroom

Official Reference Links

- AWS Architecture Center: <https://aws.amazon.com/architecture/>
- Azure Architecture Center: <https://learn.microsoft.com/en-us/azure/architecture/>
- Google Cloud Architecture Center: <https://docs.cloud.google.com/architecture>
- Apache Hadoop MapReduce Tutorial:
<https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>
- Apache Spark Structured Streaming: <https://spark.apache.org/docs/latest/streaming/index.html>
- Microsoft Threat Modeling Tool releases: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-releases>
- OWASP Threat Dragon: <https://owasp.org/www-project-threat-dragon/>
- StackShare (public company tech-stack disclosures): <https://stackshare.io/>